

SEMESTER GENAP
TA.2020 / 2021

TGL	BLN	THN
05	03	2021

		MODUL PRAKTIKUM
		SISTEM OPERASI TIM DOSEN



DIBUAT	DIPERIKSA	DISAHKAN
DOSEN / INSTRUKTUR  <u>Sigit Anggoro, S.T., M.T</u>	KETUA KELOMPOK KEAHLIAN  <u>Fajri R. Umbara, S.T., M.T.</u>	KETUA JURUSAN  <u>Wina Witanti, S.T., M.T.</u>
NID. 412103969	NID. 412185888	NID. 412176273

LABORATORIUM JURUSAN INFORMATIKA

TERAKREDITASI "B" SK.DIRJEN DIKTI: 8304/SK/BAN-PT/Ak-PPJ/S/XII/2020J
 Jl. Terusan Jend. Sudirman, PO BOX 148 Cimahi Selatan, Kota Cimahi, Jawa Barat 40531
<http://if.unjani.ac.id> | e-mail: informatika@unjani.ac.id



Modul 1

Instalasi Sistem Operasi Linux

I. Tujuan

Praktikum pada modul ini memiliki tujuan sebagai berikut

1. Peserta dapat melakukan instalasi sistem operasi pada target *device*
2. Mampu menjalankan instalasi melalui *Graphic User Interface* (GUI) maupun *Command Line* Linux
3. Mampu menganalisis proses instalasi

II. Dasar Teori

Sistem Operasi adalah perangkat lunak komputer atau software yang bertugas untuk melakukan kontrol dan manajemen perangkat keras dan juga operasi-operasi dasar sistem, termasuk menjalankan software aplikasi seperti program-program pengolah data yang bisa digunakan untuk mempermudah kegiatan manusia. Sistem Operasi berfungsi sebagai penghubung antara lapisan hardware dan lapisan software. selain itu, Sistem Operasi komputer juga melakukan semua perintah penting dalam komputer, serta menjamin aplikasi-aplikasi yang berbeda fungsinya dapat berjalan lancar secara bersamaan tanpa hambatan. Sistem operasi mempunyai penjadwalan yang sistematis mencakup perhitungan penggunaan memori, pemrosesan data, penyimpanan data dan sumber daya lainnya.

Linux merupakan sistem operasi yang dapat dikendalikan oleh salah satu atau lebih antarmuka baris perintah (*Command Line Interface* atau *CLI*) berbasis teks, antarmuka pengguna grafis (*Graphics User Interface* atau *GUI*, yang umumnya merupakan konfigurasi bawaan untuk versi desktop). Inti dari linux sendiri adalah KERNEL (bagian inti dari sistem operasi). Salah satu system operasi linux yaitu Ubuntu.

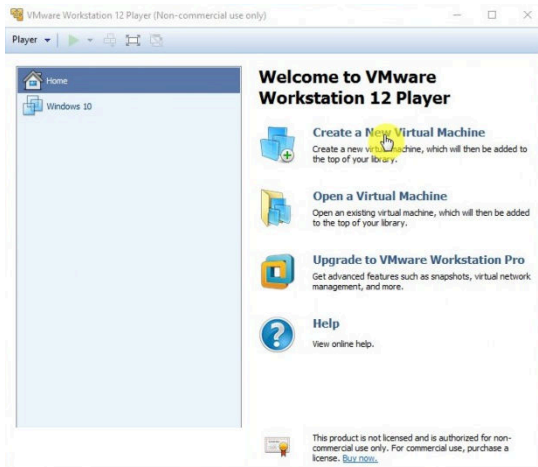
III. Langkah – Langkah

1. Persiapkan file ISO Ubuntu, bisa didownload dari <https://www.ubuntu.com/download/desktop>
2. Buka VMware > Create New Virtual Machine

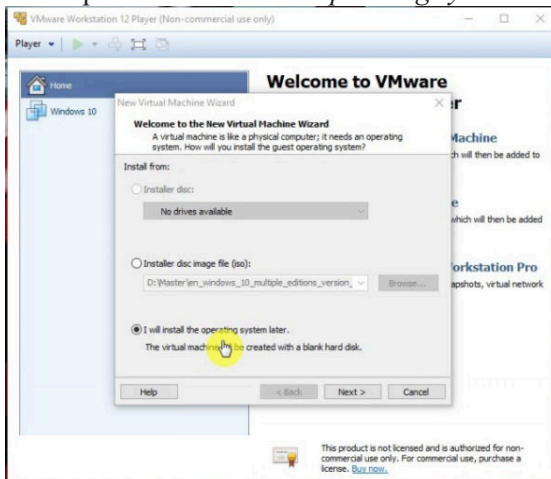


MODUL SISTEM OPERASI

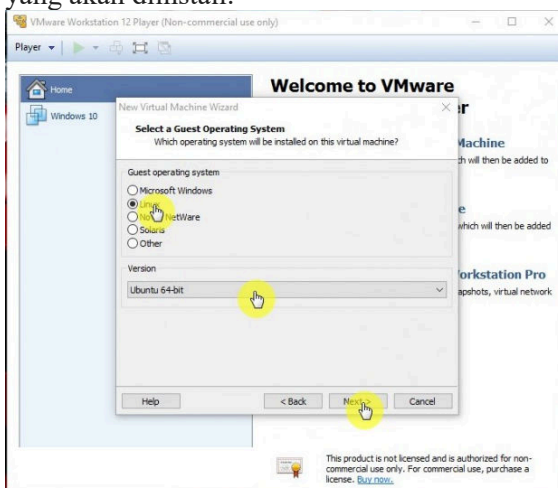
OLEH : TIM DOSEN



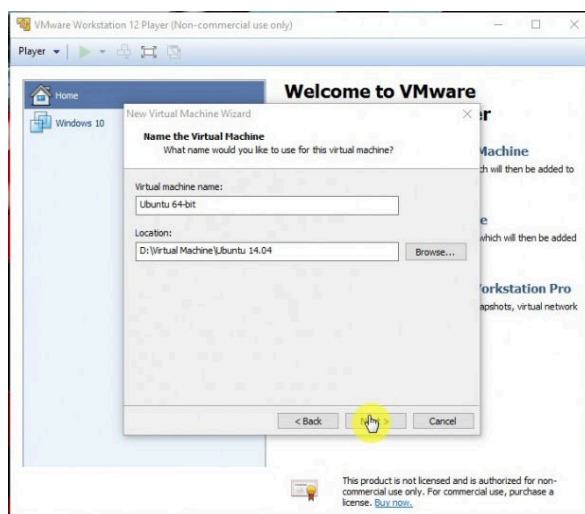
3. Pilih opsi *I will install the operating system later.*



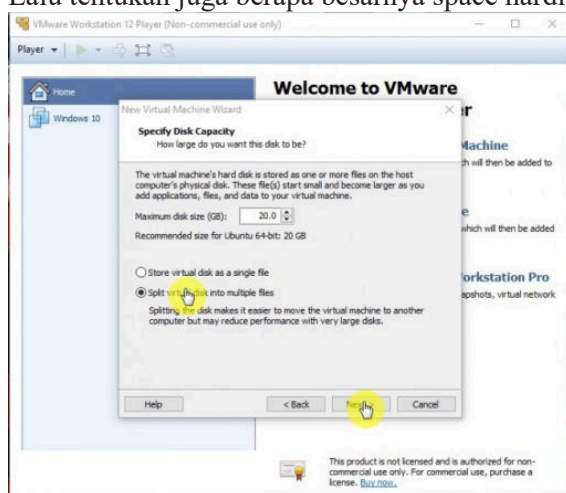
4. Pilih Linux dengan Ubuntu sebagai sistem operasinya. Untuk bit, sesuaikan dengan bit Ubuntu yang akan diinstall.



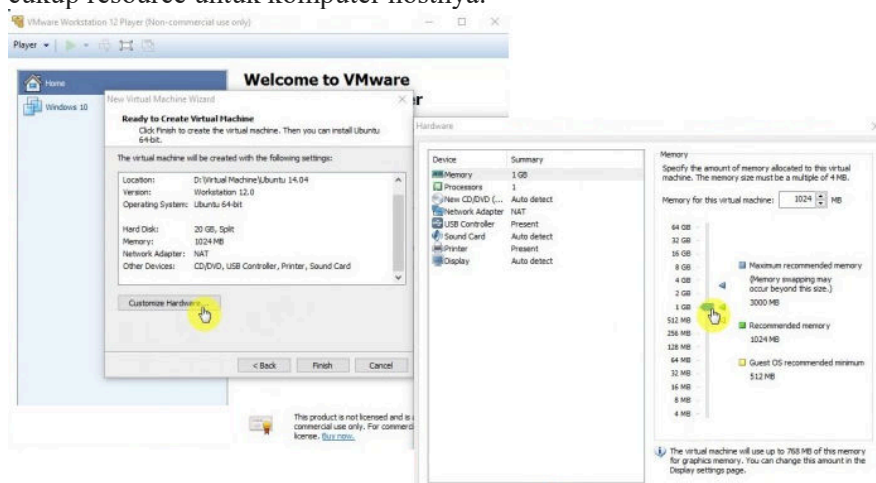
5. Setelah itu tentukan nama Virtual Machine beserta lokasi penyimpanannya.



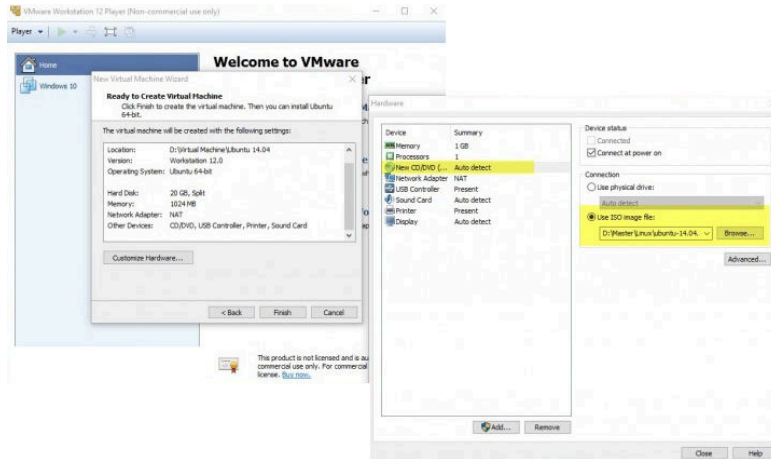
6. Lalu tentukan juga berapa besarnya space hardisk yang akan digunakan di Ubuntu.



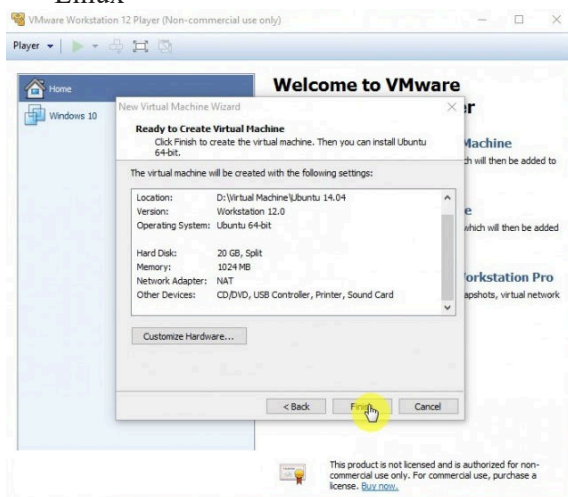
7. Klik **Customize Hardware** untuk mengkonfigurasi hardware. Kita bisa mengatur besarnya RAM virtual machine dengan cara menggeser slider yang ada. Pastikan kita menyisakan cukup resource untuk komputer hostnya.



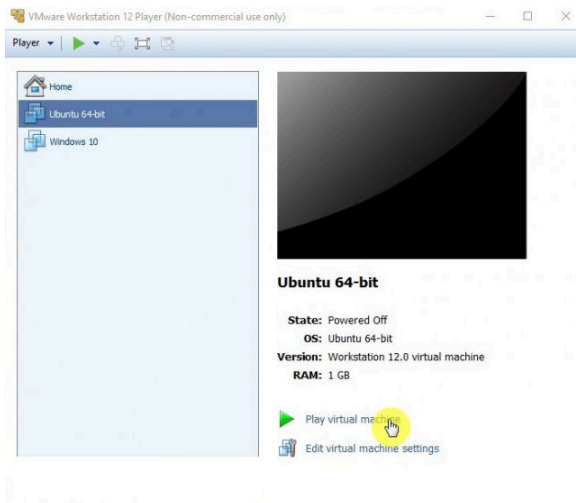
8. Pada opsi CD/DVD, pilih Use ISO image file, dan browse ke file ISO Ubuntu yang kita miliki.



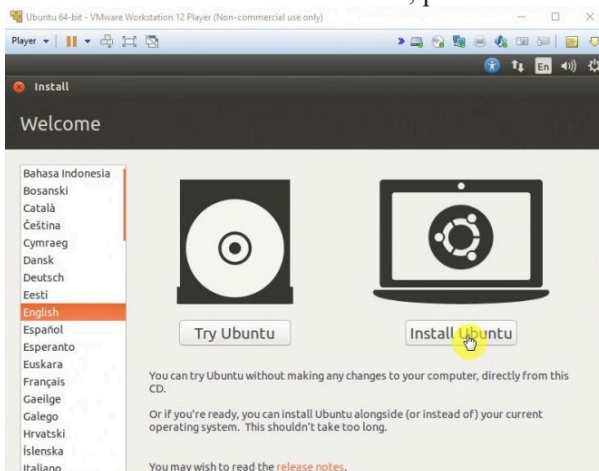
9. Jika sudah, klik Finish. Maka Virtual Machine kita sudah jadi dan siap diinstal dengan Ubuntu Linux



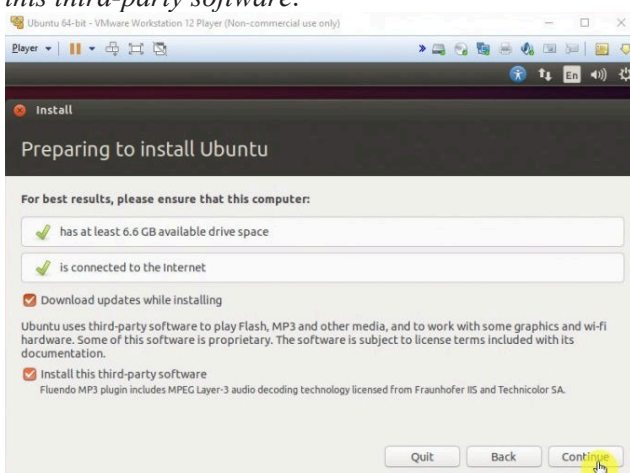
10. Play Virtual Machine yang baru saja kita buat tadi untuk mulai melakukan proses instalasi Ubuntu di VMware.



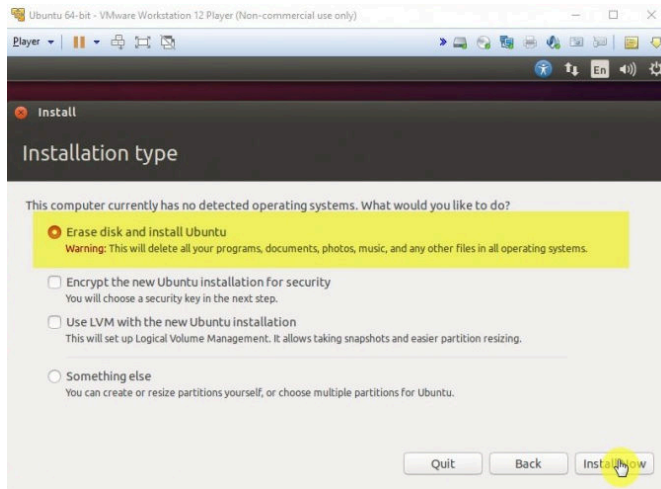
11. Setelah masuk ke installer Ubuntu, pilih Install Ubuntu.



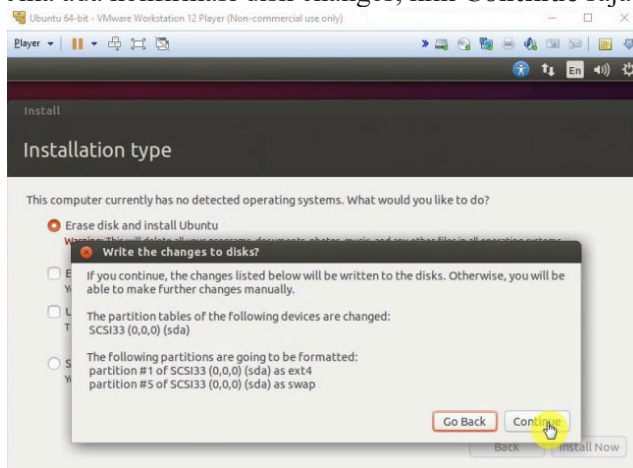
12. Jika terhubung dengan internet, kamu bisa memilih *Download updates while installing* dan *Install this third-party software*.



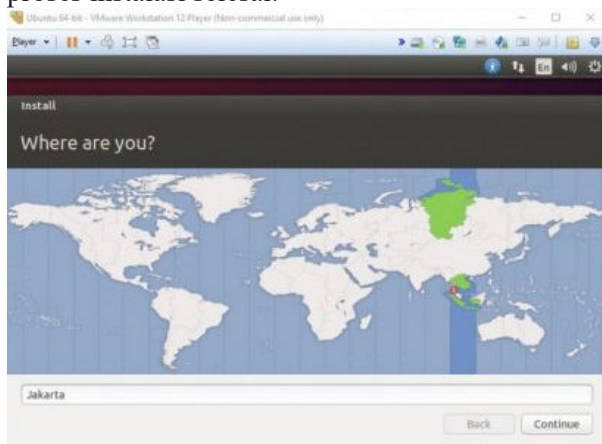
13. Pilih *Erase disk and install Ubuntu*, lalu klik **Install Now**



14. Jika ada konfirmasi disk changes, klik **Continue** saja.



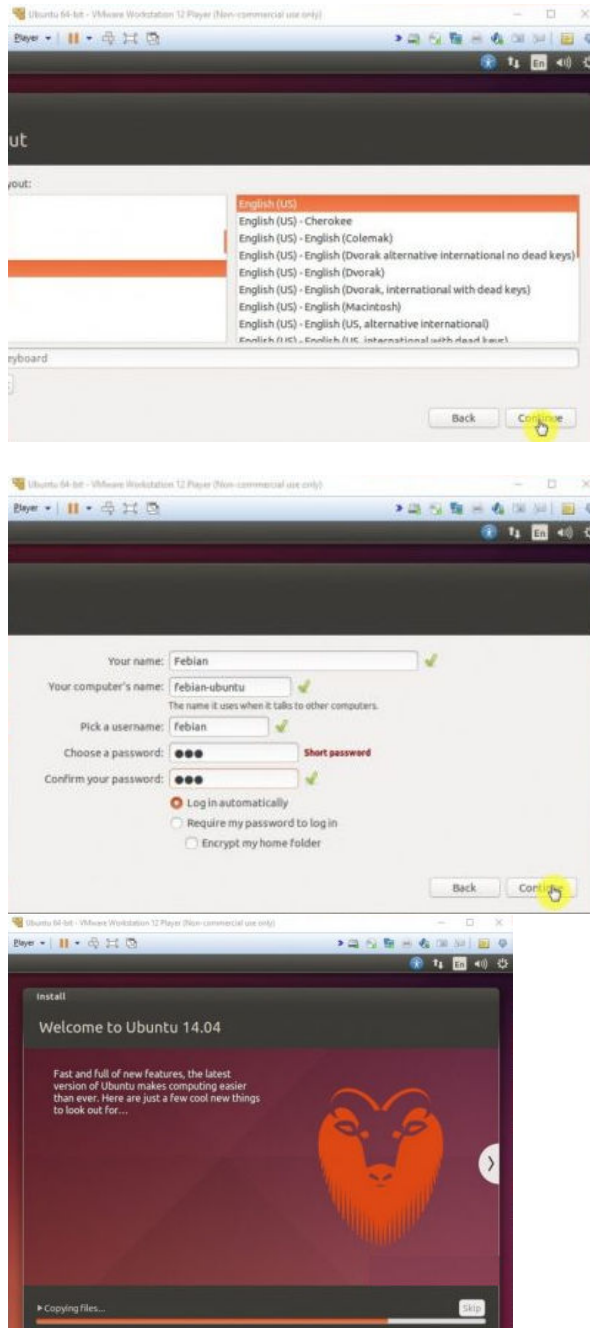
15. Sekarang proses instalasi sudah dimulai, kita tinggal ikuti saja wizard nya dan tunggu sampai proses instalasi selesai.



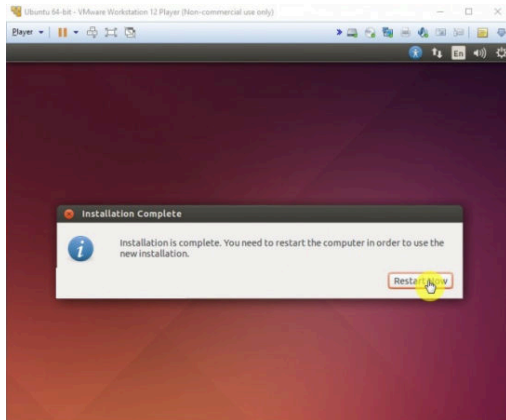


MODUL SISTEM OPERASI

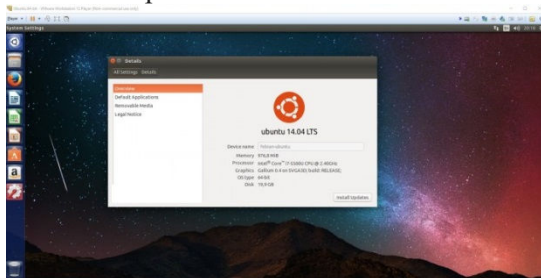
OLEH : TIM DOSEN



16. Setelah proses instalasi selesai, restart Ubuntu tersebut.



17. Berikut tampilan halaman Ubuntu setelah selesai diinstal



IV. Prosedur Praktikum

Lakukan proses Instalasi sesuai dengan langkah – langkah !

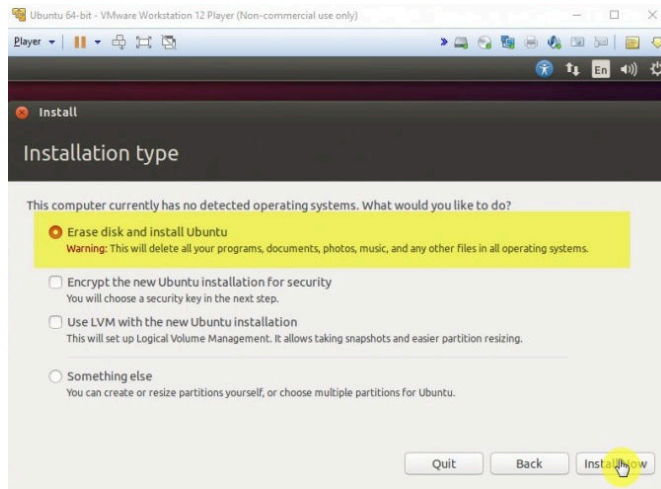
V. Tugas

1. Analisislah pada gambar kenapa saat instalasi perlu dipilih “**Erase disk and install Ubuntu**” pada kotak dialog Installation Type ?



MODUL SISTEM OPERASI

OLEH : TIM DOSEN



2. Berikan penjelasan tentang **ext4, ext3, swap, ntfs, fat32, btrfs** !
3. Buat laporanya !

Modul 2

Perintah Dasar Linux

I. Tujuan

1. Mengetahui sistem operasi GNU/Linux
2. Memahami perintah-perintah dasar GNU/Linux
3. Mampu mengoperasikan GNU/Linux pada mode CLI

II. Dasar Teori

Command line atau baris perintah adalah instruksi-instruksi yang disediakan oleh OS yang dieksekusi dari terminal dengan cara mengetikkan perintah dan diakhiri “enter”. Meskipun dewasa ini GNU/Linux sudah memiliki desktop environment yang menawan, contohnya GNOME dan KDE, akan tetapi command line tidak bisa ditinggalkan. Command line merupakan cara yang lebih efisien untuk melakukan suatu pekerjaan, oleh karena itu pengguna GNU/Linux masih menggunakan cara ini untuk bekerja. Oleh karena itu pada praktikum kali ini diawali dengan perintah dasar sebagai dasar pada untuk keseluruhan praktikum sistem operasi.

Untuk menjalankan perintah pada terminal ketik perintah yang akan dijalankan dalam huruf kecil kemudian tekan tombol Enter atau Return untuk menjalankannya.

Berikut adalah contoh perintah dasar yang sering dipakai oleh pengguna linux.

```
pwd

sysadmin@localhost:~$ pwd

/home/sysadmin

sysadmin@localhost:~$
```

Penjelasan

Direktori yang berfungsi adalah direktori tempat jendela terminal anda "saat ini". Ini juga disebut direktori saat ini. Ini akan menjadi penting ketika menjalankan perintah di masa depan karena mereka akan berperilaku berbeda berdasarkan direktori saat ini. Output dari perintah pwd (/ home / sysadmin pada contoh di atas) disebut path. Slash pertama mewakili direktori root, level teratas dari struktur direktori. Dalam output di atas, home adalah direktori di bawah direktori root dan sysadmin adalah direktori di bawah direktori home.

contoh :

```
sysadmin@localhost:~$ ls -l /etc/hosts

-rw-r--r-- 1 root root 150 Jan 22 15:18 /etc/hosts
```

rw-r--r--	Ini mewakili izin file. Izin dibahas di praktek nanti.
1	Ini mewakili sesuatu yang disebut hard link count (dibahas nanti).
root	Pemilik pengguna file.
root	Pemilik grup dari file tersebut.
150	Ukuran file dalam byte
Jan 22 15:18	Tanggal / waktu kapan file terakhir diubah.

Contoh diatas “ls” merupakan perintah untuk menampilkan konten dari sebuah direktori aktif, dengan opsi “-l” (use a long listing format). Untuk memudahkan pengguna dalam menggunakan perintah baris disediakan pula sebuah bantuan yang berisi informasi lengkap tentang perintah dan opsi-opsinya. Cara untuk mengakses bantuan terhadap sebuah perintah sebagai berikut.

contoh:

```
praktikum:$ man ls
LS(1)                User Commands                LS(1)
NAME
    ls - list directory contents
SYNOPSIS
    ls [OPTION]... [FILE]...
DESCRIPTION
    List information about the FILEs (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort.
    Mandatory arguments to long options are mandatory for short options too.
    -a, --all do not ignore entries starting with .
.....
```

Syntax Perintah Linux

Secara umum perintah-perintah Linux dan UNIX memiliki sintaks sebagai berikut:
Perintah [option...] [argumen...]

Keterangan :

- Option merupakan pilihan yang dapat kita gunakan untuk memberikan hasil tertentu dari suatu perintah.
- Argumen merupakan sesuatu yang akan diproses oleh perintah, misalnya nama file atau nama direktori.
- Tanda kurung siku ([]) merupakan simbol bahwa option dan argumen tidak harus selalu digunakan dalam menjalankan perintah.
- Tanda titik-titik (&) menandakan bahwa baik option maupun argumen dapat lebih dari satu.

Note : Semua perintah dalam Linux bersifat case sensitive, artinya huruf besar dan kecil berbeda artinya. Jadi LS akan dianggap berbeda dengan ls.

Virtual Terminal

Beberapa user dapat melakukan login pada sebuah PC atau seorang user dapat melakukan beberapa kali login yang sama pada sebuah PC. Hal ini dapat dilakukan dengan menggunakan terminal virtual. Untuk berganti terminal digunakan tombol : Alt+Fx, x adalah nomor terminal.

Contoh : jika saat ini anda berada pada mode teks dan pada terminal 1 (F1), maka untuk pindah ke terminal lainnya, tekan tombol:

Alt+F2 : pindah ke terminal 2

Alt+F3 : pindah ke terminal 3

Alt+F4 : pindah ke terminal 4

Alt+F5 : pindah ke terminal 5

Alt+F6 : pindah ke terminal 6

Alt+F7 : pindah ke mode grafik. Pada mode teks, hal ini tidak dapat berfungsi kecuali mode grafik (X window) sudah diaktifkan.

III. Prosedur Praktikum

1. Login ke sistem GNU/Linux kemudian buka terminal.

Contoh :

praktikum-PC login: azzam

Password:

.....

.....

azzam@praktikum-PC:~\$

Logout

Untuk keluar dari user yang sedang login, anda dapat mengetikkan perintah logout

Contoh:

azzam@praktikum-PC:~\$ logout

2. Pada mode CLI, lakukan praktek dan isi tabel berikut :

Perintah	Sintaks	Penggunaan
tty		
whatis		
echo echo \$home echo ~ ~sysadmin ~root ~mail ~nobody		
uname		
mkdir		



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

cd	cd / cd ~	
	cd ../dict cd /usr/share/doc	
Ls		
ls -R /etc/udev ls -d /etc/s* ls -d /etc/???? ls -d /etc/[abcd]*		
touch grep cat		
• date • cal		
• rm • rmdir • hostname • mv • cp • more • locate •		

IV. Tugas

A. Kerjakan tugas-tugas berikut

1. Jelaskan output dari perintah berikut

```
ls
cp /etc/hosts hosts
ls
```

2. Jelaskan output dari perintah berikut



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

```
rm hosts
ls
cp -v /etc/hosts hosts
ls
```

3. Jelaskan output dari perintah berikut

```
rm hosts
ls
cp -v /etc/hosts .
ls
```

4. Jelaskan output dari perintah berikut

```
rm hosts
ls
cd /etc
ls -l hosts
cp -p hosts /home/sysadmin
cd
ls -l hosts
```

5. Jelaskan output dari perintah berikut

```
rm hosts
cp -p /etc/hosts ~
cp hosts newname
ls -l hosts newname
rm hosts newname
```

6. Jelaskan output dari perintah berikut



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

```
mkdir Myetc  
cp -R /etc/udev Myetc  
ls -l Myetc  
ls -lR Myetc
```

7. Jelaskan output dari perintah berikut

```
ls  
rm -r Myetc  
ls
```

8. Jelaskan output dari perintah berikut

```
touch premove  
ls  
mv premove postmove  
ls  
rm postmove
```

B. Buat laporan mengenai penjelasan fungsi dari perintah – perintah yang telah anda praktekan !

Modul 3

Organisasi File di Linux

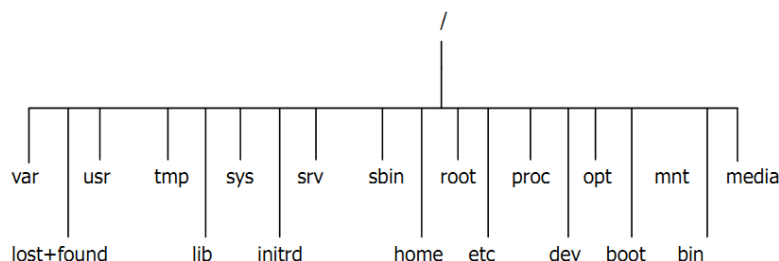
I. Tujuan

1. Mengetahui organisasi File di Linux
2. Mengetahui dan manipulasi direktori
3. Mengetahui izin akses (permission) dari file dan direktori

II. Dasar Teori

Organisasi File

Sistem file pada Linux menyerupai pepohonan (tree), yaitu dimulai dari root, kemudian direktori dan sub direktori. Sistem file pada Linux diatur secara hirarkhikal, yaitu dimulai dari root dengan symbol “/”.



Kita dapat menciptakan File dan Direktori mulai dari root ke bawah. Direktori adalah file khusus, yang berisi nama file dan INODE (pointer yang menunjuk ke data / isi file tersebut). Secara logika, Direktori dapat berisi File dan Direktori lagi (disebut juga Subdirektori).

Properti File

File mempunyai beberapa atribut, antara lain :

- Tipe file : menentukan tipe dari file, yaitu :

Karakter	Arti
-	File biasa
d	Direktori
l	Symbolic link
b	Block special file
c	Character special file
s	Socket link
p	FIFO

- Ijin Akses : menentukan hak user terhadap file ini
- Jumlah link : jumlah link untuk file ini.
- Pemilik (Owner) : menentukan siapa pemilik file ini
- Group : menentukan group yang memiliki file ini
- Jumlah karakter : menentukan ukuran file dalam byte



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

- Waktu pembuatan : menentukan kapan file terakhir dimodifikasi
- Nama file : menentukan nama file yang dimaksud

Contoh :

-rw-r--r--	1	root	root	5	2011-05-26 03:36	f2
↓	↓	↓	↓	↓	↓	↓
tipe file	izin akses	jumlah link	pemilik	group	jumlah karakter	waktu
						nama file

Nama File

Nama file maksimal terdiri dari 255 karakter berupa alfanumerik dan beberapa karakter spesial yaitu garis bawah, titik, koma, dan lainnya kecuali spasi dan karakter "&", ";", "|", "?", "\", "'", "\"", ":", "[", "]", "(", ")", "\$", "<", ">", "{", "}", "^", "#", "\\", "/". Linux membedakan huruf kecil dengan huruf besar (case sensitive). Contoh nama file yang benar :

Abcde5434

3

prog.txt

PROG.txt

Prog.txt,old

report_101,v2.0.1

5-01.web.html

Izin Akses

Setiap obyek pada Linux harus mempunyai pemilik, yaitu nama pemakai Linux (account) yang terdaftar pada /etc/passwd.

Ijin akses dibagi menjadi 3 peran yaitu :

- Pemilik (Owner)
- Kelompok (Group)
- Lainnya (Others)

Setiap peran dapat melakukan 3 bentuk operasi yaitu :

Pada File :

- R (Read) : Ijin untuk membaca
- W (Write) : Ijin untuk mengubah / membuat
- X (Execute): Ijin untuk menjalankan program

Pada Direktori :

- R (Read) : Ijin untuk membaca daftar file dalam direktori
- W (Write) : Ijin untuk mengubah/membuat file di direktori
- X (Execute): Ijin untuk masuk ke direktori (cd)



OLEH : TIM DOSEN

**Modul 4****Mengelola *Packages* dan *Processes*****I. Tujuan**

1. Mengidentifikasi bagaimana sistem file / proc digunakan oleh kernel.
2. Menggunakan perintah ps untuk melihat informasi proses
3. Mempelajari cara mengelola proses dengan memulai, menghentikan, dan melanjutkannya.

II. Dasar Teori

Dalam tugas ini, Anda akan menjelajahi direktori / proc dan perintah yang berkomunikasi dengan kernel Linux. Direktori / proc tampaknya merupakan direktori biasa, seperti / usr atau / etc, tetapi tidak. Tidak seperti / usr atau / etc/directories, yang biasanya ditulis ke disk drive, direktori / proc adalah sistem file pseudo yang disimpan dalam memori komputer.

Direktori / proc berisi subdirektori untuk setiap proses yang berjalan pada sistem. Program seperti ps dan top membaca informasi tentang menjalankan proses dari direktori ini. Direktori / proc juga berisi informasi tentang sistem operasi dan perangkat kerasnya dalam file seperti / proc / cpuinfo, / proc / meminfo dan / proc / devices.

Sub direktori / proc / sys berisi file pseudo yang dapat digunakan untuk mengubah pengaturan kernel yang sedang berjalan. Karena file-file ini bukan file "nyata", editor tidak boleh digunakan untuk mengubahnya; alih-alih, Anda harus menggunakan perintah echoor sysctl untuk mengisi isi file-file ini. Untuk alasan yang sama, jangan mencoba untuk melihat file-file ini di editor, tetapi gunakan perintah cat atau sysctl sebagai gantinya.

III. Prosedur Praktikum**A. Kernel dan /proc**

Gunakan `cat` dan kemudian `ps` untuk melihat informasi tentang proses /sbin/init (Process IDentifier (PID))

```
cat /proc/1/cmdline; echo  
ps -p 1
```

tuliskan outputnya

Ketik perintah `/proc/cmdline` untuk melihat argumen-argumen yang di-pass ke kernel saat boot time:

```
cat /proc/cmdline
```

capture hasilnya

B. Megelola Processes

Dari terminal, ketikkan perintah berikut dan capture hasilnya :
Step-1

```
ping localhost > /dev/null
```

Step-2

Hentikan proses di foreground process dengan menekan **Ctrl-C**.

Step-3

Mulai dengan proses yang sama di background, ketik :

```
ping localhost > /dev/null &
```

Step-4

Untuk melihat perintah mana yang sedang dijalankan, ketik :

```
jobs
```

Step-5

Jalankan perintah `ping` di background yang lain :

```
ping localhost > /dev/null &
```

Step-6

Sekarang 2 perintah `ping` berjalan di background. Ketik perintah `jobs` sekali lagi:

```
jobs
```

Step-7

Once you have verified that two `ping` commands are running, bring the first command to the foreground by typing the following:

```
fg %1
```

Step-8

Notice that, once again, the `ping` command has taken control of the terminal. To suspend (pause) the process and regain control of the terminal, type **Ctrl-Z**:

Step-9



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

To have this process continue executing in the background, execute the following command:

```
bg %1
```

Step-10

Issue the `jobs` command again to verify two running processes :

Step-11

Next, start one more `ping` command by typing the following:

```
ping localhost > /dev/null &
```

Step-12

Issue the `jobs` command again to verify three running processes :

Step-13

Using the job number, stop the last `ping` command with the `kill` command and verify it was stopped executing the `jobs` command:

```
kill %3  
jobs
```

Step-14

Finally, you can stop all of the `ping` commands with the `killall` command. After executing the `killall` command, wait a few moments, and then run the `jobs` command to verify that all processes have stopped:

```
killall ping  
jobs
```

C. Menggunakan `top` untuk melihat Proses

Dalam tugas ini, Anda akan menggunakan perintah `top` untuk bekerja dengan proses. Secara default program `top` mengurutkan proses dalam descending order untuk penggunaan CPU, sehingga program tersibuk akan berada di bagian atas dari daftar yang ditampilkan

step-1

Dari terminal, ketik 2 perintah berikut :

```
ping localhost > /dev/null &
ping localhost > /dev/null &
```

step-2

```
top
```

step-3

Perintah `top` adalah program interaktif, yang berarti Anda dapat mengeluarkan perintah di dalam program. Anda akan menggunakan perintah `top` untuk membunuh proses `ping`. Pertama-tama ketik huruf `k`. Perhatikan prompt telah muncul di bawah `swap`:

step-4

Di `PID to kill ]`: prompt ketikkan PID dari proses `ping` yang berjalan pertama, lalu tekan `Enter`.

step-5

Pada `Kill PID with signal [15]`: prompt, masukkan sinyal untuk mengirim ke proses ini. Dalam hal ini, cukup tekan tombol **Enter** untuk menggunakan sinyal default. Perhatikan bahwa perintah `ping` pertama dihapus dan hanya satu perintah `ping` yang tersisa dalam daftar (Anda mungkin perlu menunggu beberapa detik saat perintah `top` disegarkan):

step-6

Pertimbangkan ini

Ada beberapa nilai numerik berbeda yang dapat dikirim ke suatu proses. Ini adalah nilai yang telah ditentukan, masing-masing dengan makna yang berbeda. Jika Anda ingin mempelajari lebih lanjut tentang nilai-nilai ini, ketikkan `man kill` di jendela terminal.

Prompt menunjukkan bahwa sinyal default adalah sinyal terminasi yang ditunjukkan by `SIGTERM` atau angka 15.

step-7

Ketik `q` untuk keluar dari perintah `top`. Layar berikut mencerminkan bahwa kedua perintah `ping` diakhiri:

D. Menggunakan `pkill` dan `kill` untuk menterminasi proses

Anda akan menggunakan `pkill` dan `kill` untuk menterminate proses.

Step-1

Perintah `sleep` digunakan untuk menghentikan sejenak (pause) sebuah program (shell script) untuk waktu yang ditentukan. *In this case it is used just to provide a command that will take a long time to run.*

```
sleep 888888 &
sleep 888888 &
```

Step-2



```
jobs
```

Step-3

```
kill PID  
jobs
```

Step-4

```
pkill -15 sleep
```

E. Using ps to Select and Sort Processes

Perintah `ps` dapat digunakan untuk melihat proses. Secara default, perintah `ps` hanya akan menampilkan proses yang berjalan di shell saat ini.

Step-1

Mulai dengan background process menggunakan `ping` dan lihat current processes menggunakan `ps` command:

```
ping localhost > /dev/null &  
ps
```

Step-2

Jalankan `ps` command menggunakan opsi `-e`,

```
ps -e
```

Step-3

Gunakan `ps` command dengan opsi `-o`.

```
ps -o pid, tty, time, %cpu, cmd
```

Step-4

```
ps -o pid, tty, time, %mem, cmd --sort %mem
```

Amati perubahan di layar

Step-5



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

Perintah `ps` command daapt menunjukkan *percentage of memory* yang digunakan oleh sebuah proses, sedangkan perintah `free` akan menunjukkan *overall system memory usage*:

```
free
```

Step-7

Stop perintah `ping` dengna perintah `kill` dan verifikasi dengan perintah `jobs` :

```
kill PID  
jobs
```

IV. Tugas

1. Pada setiap langkah harus dicapture, kemudian dibahsa satu-satu
2. Buatlah laporan lengkap dan berikan kesimpulan untuk praktikan modul ini



Modul 5

Proses Input Output Di Linux

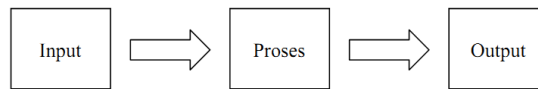
I. Tujuan

1. Mengetahui konsep proses I/O dan redirection
2. Memahami standar input, output dan error
3. Menggunakan notasi output, append dan here document
4. Mengetahui konsep PIPE dan filter

II. Dasar Teori

PROSES I/O

Sebuah proses memerlukan Input dan Output.



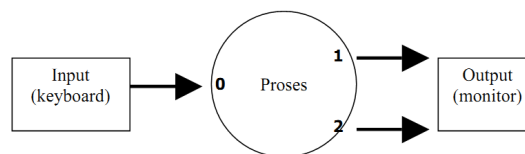
Instruksi (command) yang diberikan pada Linux melalui Shell disebut sebagai eksekusi program yang selanjutnya disebut proses. Setiap kali instruksi diberikan, maka Linux kernel akan menciptakan sebuah proses dengan memberikan nomor PID (Process Identity). Proses dalam Linux selalu membutuhkan Input dan menghasilkan suatu Output. Dalam konteks Linux input/output adalah :

- Keyboard (input)
- Layar (output)
- Files
- Struktur data kernel
- Peralatan I/O lainnya (misalnya Network)

FILE DESCRIPTOR

Linux berkomunikasi dengan file melalui file descriptor yang direpresentasikan melalui angka yang dimulai dari 0, 1, 2 dan seterusnya. Tiga buah file descriptor standar yang lalu diciptakan oleh proses adalah :

- 0 = keyboard (standar input)
- 1 = layar (standar output)
- 2 = layar (standar error)



Linux tidak membedakan antara peralatan hardware dan file, Linux memanipulasi peralatan hardware sama dengan file.

Pembelokan (Redirection)

Pembelokan dilakukan untuk standar input, output dan error, yaitu untuk mengalihkan file descriptor dari 0, 1 dan 2. Simbol untuk pembelokan adalah :

- 0< atau < pengganti standar input
- 1> atau > pengganti standar output
- 2>

PIPA (PIPELINE)

Mekanisme pipa digunakan sebagai alat komunikasi antar proses.

Input → Proses1 → Output = Input → Proses2 → Output

Proses 1 menghasilkan output yang selanjutnya digunakan sebagai input oleh Proses 2. Hubungan output input ini dinamakan pipa, yang menghubungkan Proses 1 dengan Proses2 dan dinyatakan dengan simbol “|”.

Proses1 | Proses2

Filter

Filter adalah utilitas Linux yang dapat memproses standard input (dari keyboard) dan menampilkan hasilnya pada standard output (layar). Contoh filter adalah cat, sort, grep, pr, head, tail, paste dan lainnya. Pada sebuah rangkaian pipa :

P1 | P2 | P3 | Pn-1 | Pn

Maka P2 sampai dengan Pn-1 mutlak harus utilitas Linux yang berfungsi sebagai filter. P1 (awal) dan Pn (terakhir) boleh tidak filter. Utilitas yang bukan filter misalnya who, ls, ps, lp, lpr, mail dan lainnya. Beberapa perintah Linux yang digunakan untuk proses penyaringan antara lain :

- Perintah grep
Digunakan untuk menyaring masukannya dan menampilkan baris-baris yang hanya mengandung pola yang ditentukan. Pola ini disebut regular expression.
- Perintah wc
Digunakan untuk menghitung jumlah baris, kata dan karakter dari baris-baris masukan yang diberikan kepadanya. Untuk mengetahui berapa baris gunakan option -l, untuk mengetahui berapa kata, gunakan option -w dan untuk mengetahui berapa karakter, gunakan option -c. Jika salah satu option tidak digunakan, maka tampilannya adalah jumlah baris, jumlah kata dan jumlah karakter.
- Perintah sort
Digunakan untuk mengurutkan masukannya berdasarkan urutan nomor ASCII dari karakter.
- Perintah cut
Digunakan untuk mengambil kolom tertentu dari baris-baris masukannya, yang ditentukan pada option -c.
- Perintah uniq
Digunakan untuk menghilangkan baris-baris berurutan yang mengalami duplikasi, biasanya digabungkan dalam pipeline dengan sort.

III. Prosedur Praktikum

Percobaan 1 : File Descriptor

1. Output ke layar (standar output), input dari system (kernel)

```
# ps
```

2. Output ke layar (standar output), input dari keyboard (standard input)

```
# cat
hallo, apa khabar
hallo, apa khabar
(exit dengan ^c[Ctrl-c] )
```



3. Input nama direktori, output tidak ada (membuat direktori baru), bila terjadi error maka tampilan error pada layar (standard error)

```
# mkdir mydir
```

```
# mkdir mydir (Terdapat pesan error)
```

Percobaan 2 :

1. Pembelokan standar output

```
# cat 1> myfile.txt
```

Ini adalah teks yang saya simpan

Ke file myfile.txt

2. Pembelokan standar input, yaitu input dibelokkan dari keyboard menjadi dari file

```
# cat 0< myfile.txt
```

```
# cat myfile.txt
```

3. Pembelokan standar error untuk disimpan di file

```
# mkdir mydir (Terdapat pesan error)
```

```
# mkdir mydir 2> myerror.txt
```

```
# cat myerror.txt
```

Logout dari root dan login kembali sebagai mahasiswa

4. Notasi 2>&1 : pembelokan standar error (2>) adalah identik dengan file descriptor 1.

```
$ find /var/spool/cron -print (Terdapat pesan error)
```

```
$ find /var/spool/cron -print 2> out.txt
```

```
$ cat out.txt
```

5. Notasi 1>&2 (atau >&2) : pembelokan standar output adalah sama dengan file descriptor 2 yaitu standar error

```
$ find /var/spool/cron -print 2> outx.txt 1>&2
```

```
$ cat outx.txt
```

6. Notasi >> (append)

```
$ echo "kata pertama" > surat
```

```
$ echo "kata kedua" >> surat
```

```
$ echo "kata ketiga" >> surat
```

```
$ cat surat
```

```
$ echo "kata keempat" > surat
```

```
$ cat surat
```

7. Notasi here document (<<++ ++) digunakan sebagai pembatas input dari keyboard. Perhatikan bahwa tanda pembatas dapat digantikan dengan tanda apa saja, namun harus sama dan tanda penutup harus diberikan pada awal baris

```
$ cat <<++
```

```
Hallo, apa kabar ?
```

```
Baik-baik saja ?
```

```
Ok!
```

```
++
```

```
$ cat <<%%%
```



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

Hallo, apa kabar ?

Baik-baik saja ?

Ok!

%%%

Percobaan 3 : Pipeline

```
$ who
$ who | sort
$ who | sort -r
$ who > tmp
$ sort tmp
$ rm tmp
$ ls -l /etc | more

$ ls -l /etc | sort | more
```

Percobaan 4 : Filter

```
$ w -h | grep PM
$ grep st /etc/passwd
$ ls /etc | wc -l
$ cat > kelas1.txt
Badu
Zulkifli
Yulizir
Yudi
Ade
[Ctrl-c]
$ cat > kelas2.txt
Budi
Gama
Asep
Muchlis
[Ctrl-c]
$ cat kelas1.txt kelas2.txt | sort
$ who | cut -c1-8
$ cat kelas1.txt kelas2.txt > kelas.txt
$ cat kelas.txt | sort | uniq
```

IV. Tugas

1. Masuk sebagai login root
2. Masuk ke direktori sbn, Lihat daftar secara lengkap pada direktori aktif, belokkan tampilan standard output ke file baru.
3. Lihat daftar secara lengkap pada direktori /etc/passwd, belokkan tampilan standard output ke file baru tanpa menghapus file baru sebelumnya.
4. Urutkan file baru dengan cara membelokkan standard input.
5. Urutkan file baru dengan cara membelokkan standard input dan standard output ke file baru.urut.



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

6. Buatlah direktori latihan2 sebanyak 2 kali dan belokkan standard error ke file `rmdirerror.txt`.
7. Urutkan kalimat berikut :
Jakarta
Bandung
Surabaya
Padang
Palembang
Lampung
Dengan menggunakan notasi here document (`<@@@ ...@@@`)
2. Hitung jumlah baris, kata dan karakter dari file `baru.urut` dengan menggunakan filter dan tambahkan data tersebut ke file baru.
3. Tulislah urutan perintah yang anda lakukan mulai langkah 1 sd 7 dan jelaskan maksud masing-masing perintah diatas.
4. Jalankan perintah di bawah ini dan perhatikan hasilnya serta apa maksud masing-masing perintah.
`$ cat > hello.txt`
`dog cat`
`cat duck`
`dog chicken`
`chicken duck`
`chicken cat`
`dog duck`
`[Ctrl-d]`
`$ cat hello.txt | sort | uniq`
`$ cat hello.txt | grep "dog" | grep -v "cat"`
5. Analisa hasil percobaan 1 sampai dengan 4, untuk setiap perintah jelaskan tampilannya.
6. Kerjakan tugas dari no 1 sampai dengan no 10 diatas dan analisa hasilnya.
7. Buat laporanya dan berikan kesimpulan dari praktikum ini.



MODUL SISTEM OPERASI

OLEH : TIM DOSEN



Modul 6

Konsep History

I. Tujuan

1. Mengetahui Profile
2. Mengetahui konsep history
3. Membuat dan mengeksekusi shell script sederhana
4. Mengetahui Job control

II. Dasar Teori

Profile

Pada saat login, program akan menjalankan beberapa program yaitu :

1. **/etc/profile**
Berisi shell script yang berlaku untuk seluruh pengguna Linux.
2. **Profil untuk setiap pemakai**
Pada home directory, login pertama kali akan memeriksa file `.bash_profile`. Bila tidak ada, maka file `.bash_login` akan dicari. Bila `.bash_login` tidak ada, maka dicari file bernama `.profile`.
3. **.bashrc**
File ini akan dieksekusi untuk perpindahan dari satu shell ke shell yang lain melalui instruksi su.
4. **.bash_logout**
Pada saat logout, maka bash akan mencari file `.bash_logout`. Bila ada, file tersebut akan dieksekusi sebelum logout

PATH merupakan daftar nama direktori. Bila sebuah instruksi diberikan dari prompt shell, maka instruksi tersebut akan dicari pada daftar tersebut.

PS1 adalah prompt dimana

\u = Nama User

\h = Nama Host

\W = Nama working directory

Contoh :

```
bash-3.2$ export PS1="\u@\h \w> "  
azzam@dev-db ~> cd /etc/mail  
azzam@dev-db /etc/mail>
```

History

History diadaptasi dari C-Shell, yaitu catatan dari semua instruksi yang sejauh ini telah dilakukan. Catatan ini dapat dilihat sebagai history, kemudian dapat dipilih kembali, diedit dan dieksekusi. History memudahkan pemakai untuk mengedit kembali instruksi kompleks dan panjang, terutama bila terjadi kesalahan pada penulisan instruksi maupun parameter.

Navigasi pada daftar history menggunakan karakter kontrol sebagai berikut :

^P (Ctrl-P) melihat instruksi sebelumnya

^N (Ctrl-N) melihat instruksi berikutnya

! eksekusi kembali instruksi sebelumnya

! -3 3 instruksi sebelumnya akan diulang

!88 ulangi instruksi no 88

Bash-Script

Bash-script adalah file yang berisi koleksi program yang dapat dieksekusi. Untuk eksekusi bash script gunakan `.` sebelum file bash-script yang berarti eksekusi shell dan tanda `./` berarti file bash-script berada pada direktori actual.

Job Control

Job adalah sebuah eksekusi program yang diberikan kepada kernel. Sebuah Job dianggap selesai, bila eksekusi program tersebut berakhir. Eksekusi Job adalah sama dengan eksekusi program, baik proses Background maupun proses Foreground.

III. Prosedur Praktikum

Percobaan 1 : Profile

1. File `.bash_profile` dijalankan pada home direktori pemakai yang login. File `.bash_profile` adalah hidden file, sehingga untuk melihatnya gunakan opsi `a` pada instruksi `ls`.

```
$ ls -a
$ more .bash_profile
```
2. File `.bash_logout` akan dieksekusi sesaat sebelum logout, berfungsi sebagai house clearing jobs, artinya membersihkan semuanya, misalnya menghapus temporary file atau job lainnya. Melihat file `.bash_logout` dengan instruksi

```
$ cat .bash_logout
```

Percobaan 2 : History

1. Melihat batasan maksimum instruksi yang dapat disimpan

```
$ ls -l $echo $histsize
```
2. Melihat daftar instruksi yang telah dilakukan dengan perintah history. Daftar instruksi dilengkapi dengan nomor urut. Nomor ini dapat dijadikan parameter dalam mengedit atau mengulang instruksi

```
$ history
```
3. Selain history, instruksi `fc` (fix command) juga dapat digunakan



```
$ fc -l
```

4. Instruksi fc dapat menampilkan instruksi antara 2 nomor atau dengan menggunakan kata depan instruksi yang diberikan

```
$ fc -l [Nomor urut1] [Nomor urut2] (Misalnya fc -l 85 88)
```

Percobaan 3 : Membuat Bash-script dan menjalankannya

1. Membuat file p1.sh

```
$ nano p1.sh
echo "Nama Mahasiswa: Tri Wanda Septian"
echo "Sistem Komputer"
```

2. Mengubah program menjadi executable

```
$ ls -l p1.sh
$ chmod +x p1.sh
$ ls -l p1.sh
```

3. Menjalankan script

```
$ bash p1.sh
$ sh p1.sh
$ . p1.sh
$ ./p1.sh
```

4. Konvensi dalam pembuatan script shell dinyatakan sebagai #!/bin/bash.

5. Tambahkan pada file p1.sh konvensi tersebut

```
$ nano p1.sh
#!/bin/bash
echo "Program bash script"
```

6. Buatlah file p2.sh

```
$ nano p2.sh
#!/bin/bash
echo "Program 2 bash script"
```

7. Menjalankan beberapa program shell dalam satu baris instruksi yang dipisahkan dengan tanda ;

```
$ cat p1.sh ; cat p2.sh
$ ./p1.sh ; ./p2.sh
```

8. Menjalankan script sebagai proses background, sehingga prompt tidak menunggu program tersebut selesai. Untuk menjalankan proses background gunakan tanda & pada akhir instruksi.

```
$ ./p1.sh &
$ ./p2.sh &
```

9. Untuk menjalankan 2 program atau lebih dalam satu block, kemudian dieksekusi sebagai proses background, gunakan tanda (...)

```
$ (ls ; who) > hasil &
$ cat hasil
```

Percobaan 4 : Job Control

1. Proses foreground

```
$ ps x
```

2. Proses background

```
$ ps x > hasil &
```

3. Setiap job mempunyai PID yang tunggal (unique). Untuk melihat jobs yang aktif

```
$ jobs
```

4. Buatlah file ploop.sh. File ini tidak akan pernah berhenti kecuali ditekan Ctrl-C

```
$ nano ploop.sh
#!/bin/bash
while [ true ]
```

```
do
sleep 10
echo "Hallo"
done
```

5. Buatlah file ploop.sh menjadi executable. Jalankan program, akan ditampilkan kata Hallo setiap 10 detik. Untuk keluar program, tekan Ctrl-C (^C)

```
$ chmod +x ploop.sh
$ ./ploop.sh
```
6. Jalankan program sebagai background

```
$ ./ploop.sh &
```
7. Periksa job yang aktif

```
$ jobs
```
8. Ubah job menjadi foreground, gunakan fg dan nomor job, atau fg dengan argumen berupa nama jobs yang sedang berjalan, atau bila tidak rancu dengan jobs yang lain, gunakan huruf depan nama jobs yang sedang berjalan.

```
$ fg %[Nomor Job] (Contoh : fg %1) atau
$ fg %ploop.sh
```
9. Untuk mengembalikan jobs tersebut ke background, tekan Ctrl-Z (^Z), kemudian jalankan instruksi bg

```
$ bg
$ jobs
```

Percobaan 5 : Manipulasi stack untuk Direktori

1. Instruksi dirs digunakan untuk melihat stack direktori, pada output hanya ditampilkan direktori home ~

```
$ dirs
```
2. Membuat 3 buah direktori

```
$ mkdir marketing sales support
```
3. Masukkan direktori sales ke dalam stack dengan instruksi pushd. Maka terdapat 2 direktori dalam stack yaitu \$HOME/sales dan \$HOME. Kemudian lihat direktori aktual

```
$ pushd sales
$ pwd
```
4. Masuk ke direktori support

```
$ pushd /home/mahasiswa/support
$ pwd
```
5. Lakukan kembali untuk direktori marketing

```
$ pushd ../marketing
$ pwd
```
6. Bila pushd dilakukan tanpa argumen, maka stack akan mengambil direktori berikutnya

```
$ pushd
```
7. Untuk membuat direktori sales menjadi direktori paling atas (top stack), maka pushd dapat dilakukan dengan argumen +n, dimana n adalah nomor urutan direktori tersebut

```
$ pushd +2
$ pwd
```
8. Untuk menghapus direktori dari stack, gunakan instruksi popd

```
$ popd
```



```
$ popd +2  
$ dirs
```

Percobaan 6 : Alias

1. Alias adalah mekanisme untuk memberi nama alias pada satu atau sekelompok instruksi. Untuk melihat alias yang sudah terdaftar pada system :
\$ alias
2. Membuat beberapa alias
\$ alias del='rm -i'
\$ alias h='history'
3. Gunakan instruksi hasil alias
\$ ls
\$ del hasil
\$ h | more
4. Untuk menghapus alias gunakan instruksi unalias
\$ unalias del
\$ del files (Terdapat Pesan Kesalahan)

IV. Tugas

1. Bash Script
 - a. Buat 3 buah script p1.sh, p2.sh, p3.sh dengan isi masing-masing :

```
p1.sh  
#!/bin/bash  
echo "Program p1"  
ls -l
```

```
p2.sh  
#!/bin/bash  
echo "Program p2"  
who
```

```
p3.sh  
#!/bin/bash  
echo "Program p3"  
ps x
```

- b. Jalankan script tersebut sebagai berikut :

```
$/p1.sh ; ./p3.sh ; ./p2.sh  
$/p1.sh &  
$/p1.sh $ ./p2.sh & ./p3.sh &  
$( ./p1.sh ; ./p3.sh ) &
```

2. History
 - a. Ganti nilai HISTSIZE dari 1000 menjadi 20
\$ HISTSIZE=20



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

\$ h

- b. Gunakan fasilitas history dengan mengedit instruksi baris ke 5 dari instruksi yang terakhir dilakukan

\$!-5

- c. Ulangi instruksi yang terakhir. Gunakan juga ^P dan ^N untuk bernavigasi pada history bufer

\$!!

- d. Ulangi instruksi pada history bufer nomor 150

\$!150

- e. Ulangi instruksi dengan prefix "ls"

\$!ls

3. Analisis Tugas no 1 dan 2, jelaskan maksudnya !
4. Buat laporan dan berikan kesimpulan pada praktikum ini !

**MODUL 7****Shell Programming****I. Tujuan**

1. Memahami konsep shell programming
2. Mampu menulis program dengan shell programming
3. Memahami konsep Shell interaktif

II. Dasar Teori

Shell adalah sebuah bahasa penterjemah perintah (command interpreter language) atau sebuah prosesor makro yang menjalankan perintah. Shell juga dapat berarti interpreter perintah yang menjadi antarmuka antara user dengan utilitas dan bahasa pemrograman. Dengan shell, dapat dibuat sebuah perintah atau file yang berisi perintah-perintah itu sendiri. Perintah baru tersebut mempunyai status yang sama dan di letakkan pada direktori /bin.

shell mengizinkan eksekusi perintah secara synchronously dan asynchronously. Shell menunggu perintah synchronous untuk dilengkapi sebelum menyetujui lebih banyak input, sedangkan perintah asynchronous terus berjalan dalam paralel dengan shell ketika membacanya dan menjalankan perintah tambahan. Shell juga mengenal adanya redirection. Dengan redirector dapat dilakukan kontrol untuk input dan output dari perintah yang ada dan juga melakukan kontrol terhadap isinya. Secara default, shell UNIX juga menyediakan perintah-perintah built-in, seperti pwd, cd, kill, history, atau utilitas lain yang terpisah.

Shell dapat digunakan dalam dua modus, yaitu modus interaktif dan modus skrip. Pada modus interaktif, kita menuliskan satu baris perintah disamping prompt dan mengakhirinya dengan ENTER. Hasil eksekusi akan ditampilkan langsung pada layar dan kita kembali disediakan prompt untuk memasukkan perintah-perintah berikutnya. Pada modus skrip, anda menuliskan skrip atau kode program, berisi perintah-perintah Linux, menyimpannya ke dalam sebuah file dengan ekstensi .sh. Selanjutnya file ini dijalankan pada modus Interaktif.

Jenis – jenis Shell

Pada UNIX/Linux terdapat berbagai jenis shell dengan kelebihan dan kekurangan masing-masing. Dengan banyaknya variasi shell ini, user bebas memilih shell yang digunakan. Meskipun kebanyakan sistem operasi telah menentukan sebuah shell sebagai shell default, tetapi tidak menutup kemungkinan shell lain juga dapat dijalankan. Berikut ini beberapa jenis shell yang umum terdapat dalam sistem operasi UNIX/Linux.

- Bourne Shell (/bin/sh)
- Bourne Again Shell(/bin/bash)
- C Shell (/bin/csh)
- Tenex C Shell (/bin/tcsh)
- Tcl shell (/bin/tclsh)
- Korn shell (shell /bin/ksh)
- Public domain korn shell (/bin/pdksh)
- A shell (/bin/ash)
- Z shell (/bin/zsh)

Mengganti Shell

Linux menggunakan bash sebagai shell default, tetapi pengguna bisa mengubah shell default untuk setiap user-nya. Untuk melihat shell yang sedang digunakan oleh user bisa dilihat pada file

/etc/passwd.

```
root:x:0:0:root:/root:/bin/bash
```

```
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
```

Isi dari file tersebut tiap barisnya dibagi menjadi tujuh bagian, dan setiap terakhir digunakan untuk mendefinisikan shell yang digunakan. Dalam contoh di atas user root menggunakan shell Bash sedangkan daemon menggunakan shell bourne shell. Cara lain yang bisa digunakan untuk melihat shell adalah dengan melihat environmet user dengan menjalankan perintah env. Environment user merupakan lingkungan user yang berisi semua variabel atau ketentuan khusus untuk user tersebut.

```
$env
.....
SHELL=/bin/bash
.....
```

Ada beberapa cara yang dapat digunakan untuk mengubah shell default, yaitu dengan utilitas userconf, atau dapat juga dengan mengubah pada file /etc/passwd tersebut tetapi harus sebagai root. Contoh mengganti shell dengan utilitas chsh (change shell).

```
$chsh
Password:
Changing the login shell for praktikum
Enter the new value, or press ENTER for the default
Login Shell [/bin/bash]: /bin/sh
```

Shell Interaktif

Sebuah shell interaktif dapat dijalankan tanpa harus mengubah default shell-nya. Cara yang digunakan adalah dengan memanggil nama shell pada command prompt. Misalnya, user akan menggunakan shell sh, user tinggal menjalankan shell sh, user tinggal menjalankan sh sehingga prompt akan berubah sesuai dengan prompt Bourne shell. Untuk keluar dan kembali ke shell default, ketik exit atau tekan Ctrl + d.

```
bash@praktikum:~$ sh
$
$ exit
```

Script Shell

Untuk menjalankan sebuah script shell, sebaiknya lebih dulu memahami penggunaan path absolut dan path relatif. Untuk membuatnya kita bisa menggunakan text editor dan simpan file nya dengan ekstensi .sh. Untuk menjalankan filenya kita harus merubah izin aksesnya terlebih dulu dengan



menggunakan perintah `$ chmod +x nama_file.sh $`. Setelah itu file tersebut baru bisa dijalankan dengan perintah `./nama_file.sh`

III. Prosedur Praktikum

Percobaan 1 : Shell Interaktif

Jalankan perintah – perintah di bawah ini satu per satu dan jelaskan maksudnya !

```
$ find . -name '*.jpg' $
```

```
file nama_file
```

```
$ echo "belajar shell linux" > nama_file $
```

```
echo "keren abis" >> nama_file
```

```
$ echo "OK...sepakat .... te o pe deh" >> nama_file
```

```
$ cat nama_file
```

```
$ more nama_file
```

```
$ grep keren nama_file $
```

```
man grep
```

Percobaan 2 : Script Shell

Gunakan text editor untuk membuat filenya.

Latihan1.sh

```
#!/bin/bash
```

```
echo "selamat datang $USER"
```

```
echo "di shell programming"
```

Latihan2.sh

```
#!/bin/bash
```

```
echo "Shell yang digunakan adalah $SHELL"
```

```
echo "saat ini jam `date +%T`"
```

```
echo "tanggal `date +%D`"
```

Latihan3.sh

```
#!/bin/bash
```

```
clear
```

```
echo "komputer anda telah menyala selama `uptime`"
```

```
echo "jumlah user yang login sebanyak `who | wc -l` user"
```

```
echo "anda login dengan user $LOGNAME"
```

```
echo "di shell $SHELL"
```



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

Latihan4.sh

```
#!/bin/bash
```

```
TITEL="Membuat Fungsi Sistem Informasi $HOSTNAME"
```

```
SAAT_INI=$(date+"%d %T %Z")
```

```
UPD="Sistem ini di update oleh $USER pada tanggal
```

```
$SAAT_INI"
```

```
function info_uptime(){
```

```
    echo "<h2>informasi uptime</h2>"
```

```
    echo "<pre>"
```

```
    uptime
```

```
    echo "</pre>"
```

```
}
```

```
cat <<- EOF
```

```
<HTML>
```

```
<HEAD>
```

```
<TITLE>$TITEL</TITLE>
```

```
</HEAD>
```

```
<BODY>
```

```
<H1>$TITEL</H1>
```

```
<P>$UPD</P>
```

```
$(info_uptime)
```

```
</BODY>
```

```
</HTML>
```

```
EOF
```

IV. Tugas

1. Analisa hasil percobaan 1 dan 2, untuk setiap perintah dan latihan file jelaskan maksud dari tampilannya!
2. Buat laporanya dan berikan kesimpulan dari praktikum ini !



Modul 8

Programming di Linux

I. Tujuan

1. Memahami cara membuat program di linux
2. Mengetahui cara kompilasi program di linux

II. Dasar Teori

Sistem operasi Linux adalah platform yang dapat digunakan untuk pengembangan program dan aplikasi yang mendukung penggunaan banyak bahasa seperti C, java dan C++.

Bahasa C adalah bahasa pemrograman yang sangat cocok untuk dipelajari bagi yang ingin mengetahui lebih dalam tentang Linux dan Sistem Operasi Unix lainnya yang sebagian besar dibuat dengan bahasa C. Linux sendiri merupakan sistem operasi yang langsung menyediakan C pada saat instalasi.

Tiga hal yang dibutuhkan dalam membuat program C adalah :

1. Text editor,

Text editor diperlukan untuk membuat source code program C. text editor yang bisa digunakan di linux antara lain : vi, gedit, kedit, nano dan lain sebagainya.

2. Compiler ,

Compiler adalah program yang mengkonversi source code menjadi bahasa mesin (biasa disebut object code atau machine code) sehingga dapat dimengerti langsung oleh CPU. Ada beberapa compiler yang bisa digunakan dalam sistem operasi linux, diantaranya cc, gcc dan g++. C compiler termasuk dalam GNU Compiler Collection (GCC), salah satu komponen penting dalam distribusi linux. Untuk mengecek GCC pada sistem ketikkan gcc pada command line, jika pesan yang diterima berupa : "no input files" berarti gcc telah terinstall dan siap digunakan, jika tidak pesan yang diterima adalah "command not found". Proses kompilasi dilakukan dengan menggunakan command line (shell).

3. C standard library

Library adalah kumpulan subprogram yang dapat digunakan programmer untuk membuat source code. glibc adalah standar C library yang diimplementasikan oleh GNU. Sebagian besar komponen linux membutuhkan standard tersebut.

Contoh sederhana:

```
#include <stdio.h>

Main()
{
    Printf("Informatika Unjani\n");
}
```

```
}

```

Simpan program dengan nama file coba.c

Baris pertama pada kode diatas disebut dengan preprocessor directive yang diawali dengan tanda '#'. Directive terdiri dari header file yang berada dalam glibc dan memberitahukan compiler bahwa kita menggunakan fungsi yg terdapat di header file tersebut. Contohnya, pada kode diatas directive yang digunakan adalah stdio.h yang menyediakan fasilitas input dan output pada C, sehingga kita dapat menggunakan perintah "printf".

Baris kedua adalah fungsi main(). Setiap program C memiliki fungsi main() yang berisi satu atau lebih statement yang dituliskan didalam kurung kurawal ({}). Fungsi main() adalah langkah awal untuk membuat program.

Perintah `Printf("Informatika Unjani\n");` digunakan untuk menampilkan teks Informatika Unjani dari program C ke layar. Huruf f di akhir kata printf berarti formatted (terformat). Setiap string yang ingin ditampilkan dengan perintah printf harus dibuat dalam tanda petik dua (double quotes).

Simpan kode tersebut dengan nama apapun dengan disertai extension .c, extension untuk program c yang akan dikenali oleh compiler sebagai C source code. Contohnya simpan kode diatas dengan nama coba.c.

Cara meng-compile dengan menggunakan GCC adalah dengan mengetikkan perintah di bawah ini pada command line :

```
gcc -o coba coba.c
```

Compile program akan menghasilkan file berextension .exe yaitu coba.exe. Untuk menjalankan program yang telah dcompile, caranya adalah dengan mengetikkan pada command line :

```
./coba
```

Run program diatas akan menghasilkan tulisan Informatika Unjani di layar.

III. Prosedur Praktikum

1. Perbaiki Script-Script di bawah ini, kompilasi dengan \$gcc.

Script1.c

```
#include main()
{ printf("Hello, world!\n");
return 0;
}
```

Script2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void main()
{
printf(<Files in Directory are: \> );
```



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

```
system(<ls -l>;)
}
```

Script3.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void main()
{
    int tinggi, alas, i, j;
    printf("\t\t\t====SEGITIGA BINTANG====\n");

    printf("\n");
    printf("\n");
    printf("Masukkan tinggi SEGITIGA : ");
    scanf("%i", &tinggi);
    alas = (tinggi + 1)/2;

    if(tinggi%2==0)
    {
        printf("segitiga tidak bisa dibuat,input salah ");
    }
    else
    {
        for (i=1;i<=alas;i++)
        {
            for (j=1;j<=i;j++)
                printf("*");
            printf("\n");
        }

        for(i=alas-1;i>=1;i--)
        {
            for (j=1;j<=i;j++)
                printf("*");
            printf("\n");
        }

        printf("\n");
    }
}
```

2. Dalam laporan sertakan script, script yang sudah diperbaiki, screenshot program yang sudah diperbaiki, serta
3. Jelaskan perbaikan script yang dilakukan

IV. Tugas

1. Buatlah dan jelaskan sebuah program yang dapat menghitung luas dan keliling dari segitiga, persegi panjang dan lingkaran.
2. Buatlah dan Jelaskan sebuah program yang dapat menampilkan segitiga seperti berikut:

```
*
***
*****
```

Bisa memasukkan tinggi sesuai keinginan, gambar diatas berarti segitiga mempunyai tinggi 3.

Modul 9

Proses dan Thread

V. Tujuan

1. Mengerti konsep proses dan thread
2. Mampu membuat proses dan thread
3. Mampu memanajemen proses

VI. Dasar Teori

Proses

Proses adalah program yang sedang dieksekusi. Setiap kali menjalankan suatu program, Sistem UNIX melakukan suatu `fork()`, yaitu melakukan beberapa urutan operasi untuk membuat suatu proses konteks dan kemudian mengeksekusi program tersebut dalam konteks yang sudah dibuat. Setiap proses yang dijalankan akan memiliki PID (Process ID). Kita dapat melihat proses yang berjalan dengan perintah `ps`. Dengan `fork()` sistem call akan membuat proses baru yang identik dengan proses induk kecuali proses ID. Proses dicopy ke memory dari proses parent dan strukture proses baru ditangani oleh kernel. Lingkungan, resource limits, umask, controlling terminal, direktori tempat eksekusi, root direktori, signal mask, dan resource proses lainnya digandakan dari proses induk oleh proses `fork`.

Thread

Operasi thread meliputi pembuatan thread, penghentian, sinkronisasi, penjadwalan, manajemen, dan pengulangan proses.

- Thread tidak mengatur pembuatan thread lain, atau mengetahui sebuah thread lain telah dibuat.
- Semua thread dengan sebuah proses saling berbagi pakai address space.
- Thread pada satu proses yang sama saling berbagi pakai:
 - Process instructions
 - Most data
 - open files (descriptors)
 - signals and signal handlers
 - current working directory
 - User and group id
- Setiap thread merupakan unik
 - Thread ID
 - set of registers, stack pointer
 - stack for local variables, return addresses
 - signal mask
 - priority



- Return value: errno
- fungsi pthread memberikan nilai #0# jika OK

Contoh :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *print_message_function( void *ptr );
main()
{
    pthread_t thread1, thread2;
    char *message1 = "Thread 1";
    char *message2 = "Thread 2";
    int iret1, iret2;
    /* Create independent threads each of which will
    execute function */
    iret1 = pthread_create( &thread1, NULL,
    print_message_function, (void*) message1);
    iret2 = pthread_create( &thread2, NULL,
    print_message_function, (void*) message2);
    /* Wait till threads are complete before main
    continues. Unless we */
    /* wait we run the risk of executing an exit which
    will terminate */
    /* the process and all threads before the threads
    have completed. */
    pthread_join( thread1, NULL);
    pthread_join( thread2, NULL);
    printf("Thread 1 returns: %d\n",iret1);
    printf("Thread 2 returns: %d\n",iret2);
    exit(0);
}
void *print_message_function( void *ptr )
{
    char *message;
    message = (char *) ptr;
    printf("%s \n", message);
}
```

VII. Prosedur Praktikum

Percobaan 1:

- Untuk mengetahui berapa jumlah memory yang tersisa dan yang terpakai. Gunakan perintah: 'free --okt'. Perintah tersebut akan menampilkan jumlah byte tentang memory
- Top juga menampilkan informasi memory secara kontinu. Untuk menggunakannya ketik: 'top'
- Menggunakan perintah 'at', Perintah at memungkinkan kita untuk mengantriakan eksekusi suatu perintah pada waktu tertentu. Sebagai contoh, untuk membentuk pekerjaan pada jam 2:30 a.m. Untuk menuliskan shcedule pekerjaan dengan 'at'. yakni dengan menulis perintah baris demi baris dan menekan enter untuk masing-masing perintah dan untuk mengakhiri dapat digunakan <CTRL + D>. Ikuti langkah berikut:

Ketik : \$ date

Ketik : \$ at <waktu yang mendekati waktu saat ini> misal : \$ at 17:15

Akan muncul prompt 'at'

Ketikkan : \$ dmesg > simpanan

Tekan ENTER kemudian CTRL+D

Lihatlah daftar antrian dengan mengetikkan : \$at #l

Apabila ada proses yang ingin dibatalkan, bisa dilakukan dengan cara mengetikkan :

\$ at --d nomorjobid, misal : \$at --d 12

- d) Untuk mengetahui sebelumnya apakah ada program yang sedang berjalan atau tidak, ketik: \$ jobs Atau untuk melihat proses ID '\$ jobs --p' Atau untuk melihat PID dan apa yang sedang dilakukan '\$ jobs --l'

Untuk menjalankan kembali proses yang disuspend di foreground, bisa digunakan perintah 'fg'. Ketik : '\$ fg % nomorPIDpadajobs' atau untuk proses terakhir yang diproses cukup mengetik '\$ fg' . Sekarang kita akan mencoba menjalankan proses di background atau dalam kondisi suspend. Kita akan membuat sebuah script shell yang bertujuan berjalan terus-menerus dan diletakkan di proses background. Buat file dengan nama bgproses.sh di direktori /bin.

Ketik:

```
#!/bin/sh
i=0
while [ true ];
do
i=$((i+1))
done
echo $i
```

Ubah hak aksesnya agar bisa dieksekusi. Ketik: '\$ chmod +x bgproses.sh' Jalankan script yang dibuat. Ketik: '\$ bgproses.sh' Tekan CTRL+Z untuk melakukan suspend pada proses yang sedang berjalan.

Ketik perintah : '\$ ps' Apakah ada proses bgproses.sh? berapa PID-nya? Karena sedang disuspend, maka bgproses.sh dalam kondisi stopped.

Untuk menjalankan kembali proses bgproses.sh, Ketik :

'\$ bg % nomorPIDpadajobs'

Lihat menggunakan jobs, apakah bgproses.sh telah berjalan kembali. Untuk mematikan secara permanen proses yang sedang berjalan. Kita bisa menggunakan perintah kill. Caranya:

'\$ kill -9 <NoPIDmakebgNPM.sh>'

Percobaan 2: Cobalah program berikut

```
#include <stdio.h> /* printf, stderr, fprintf */
#include <unistd.h> /* _exit, fork */
#include <stdlib.h> /* exit */
#include <errno.h> /* errno */

int main(void)
{
    pid_t pid;

    /* Output from both the child and the parent process
     * will be written to the standard output,
     * as they both run at the same time.
     */
    pid = fork();
```



```
if (pid == 0)
{
    /* Child process:
     * When fork() returns 0, we are in
     * the child process.
     * Here we count up to ten, one each second.
     */
    int j;
    for (j = 0; j < 10; j++)
    {
        printf("child: %d\n", j);
        sleep(1);
    }
    _exit(0); /* Note that we do not use exit() */
}
else if (pid > 0)
{
    /* Parent process:
     * When fork() returns a positive number, we are in the parent process
     * (the fork return value is the PID of the newly- created child process).
     * Again we count up to ten.
     */
    int i;
    for (i = 0; i < 10; i++)
    {
        printf("parent: %d\n", i);
        sleep(1);
    }
    exit(0);
}
else
{
    /* Error:
     * When fork() returns a negative number, an error happened
     * (for example, number of processes reached the limit ).
     */
    fprintf(stderr, "can't fork, error %d\n", errno);
    exit(EXIT_FAILURE);
}
}
```

VIII. Tugas

1. Analisis Percobaan 2, jelaskan maksud tampilan dari program tersebut !
2. Pahamiilah kode hello.c berikut : D

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_THREADS 5
void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
```



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

```
printf("Hello World! It's me, thread #%ld!\n", tid);
pthread_exit(NULL);
}
int main(int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0;t<NUM_THREADS;t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    /* Last thing that main() should do */
    pthread_exit(NULL);
}
```

kemudian eksekusi juga dengan beberapa opsi berikut, tuliskan apa yang terjadi.

icc -pthread -o hello hello.c

pathcc -pthread -o hello hello.c

pgcc -lpthread -o hello hello.c

gcc -pthread -o hello hello.c

3. Buat laporan dan berikan kesimpulan dari praktikum ini !



MODUL SISTEM OPERASI

OLEH : TIM DOSEN

Modul 10

IMPLEMENTASI SISTEM OPERASI

Ujian Akhir Semester untuk praktikum Sistem Operasi. Bahasan ujiannya dari semua modul.